



6. Bounded Contexts



Creo que este documento es uno de los más importantes de toda la arquitectura. Hasta ahora definimos **qué existe** (Domain Model), **qué significa** (Ubiquitous Language), **cómo se organiza** (Architectural Elements) y **qué capacidades tendrá el sistema** (Engines). Ahora debemos definir **hasta dónde llega la responsabilidad de cada parte del dominio**.

En otras palabras:

- **Engine** = capacidad del sistema.
- **Component** = módulo funcional.
- **Bounded Context** = frontera semántica donde un modelo tiene un único significado.

No son lo mismo. Un Engine puede contener uno o varios Bounded Contexts, y un Bounded Context puede contener varios Components.

Objetivo

El Personal Operating System organiza un dominio amplio que incluye trabajo, conocimiento, aprendizaje, recursos y evolución personal.

Para mantener la consistencia del modelo y evitar ambigüedades, el dominio se divide en **Bounded Contexts**.

Cada Bounded Context define un límite conceptual dentro del cual las entidades, procesos y reglas poseen un significado único y consistente.

Los Bounded Contexts no representan herramientas, carpetas ni módulos técnicos.

Representan fronteras del lenguaje del dominio.



¿Qué es un Bounded Context?

Un Bounded Context es un límite dentro del cual:

- existe un lenguaje ubicuo consistente;
- las entidades poseen un único significado;
- las reglas del dominio son coherentes;
- los procesos operan sobre el mismo modelo conceptual.

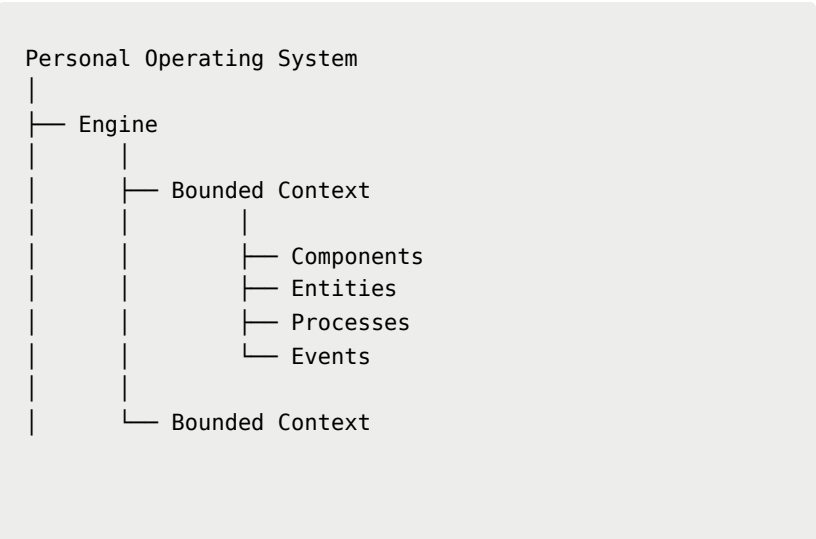
Fuera de ese límite, una entidad puede existir nuevamente, pero con otra responsabilidad.

Principios

Todo Bounded Context cumple los siguientes principios:

- posee una responsabilidad única;
- encapsula reglas del dominio;
- mantiene consistencia semántica;
- minimiza dependencias externas;
- expone capacidades mediante interfaces conceptuales;
- puede evolucionar independientemente.

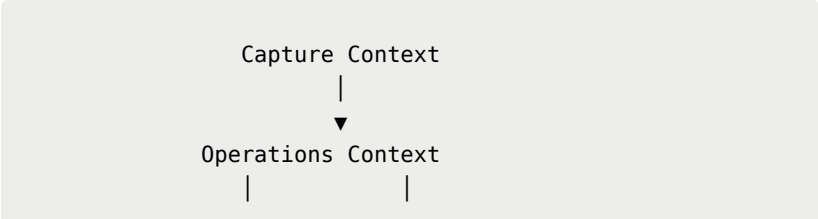
Relación con la arquitectura



Bounded Contexts del POS

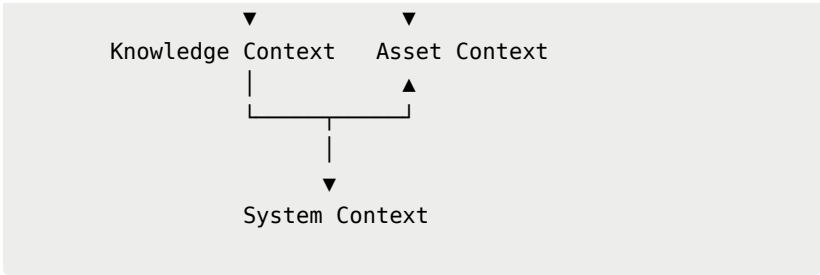
Operations Context	Knowledge Context	Capture Context	Asset Context	System Context
Responsabilidades Gestionar el trabajo operativo del sistema. Es responsable de la planificación, ejecución y seguimiento de las operaciones recurrentes y los proyectos.	Responsabilidades Gestionar la creación, validación, organización y reutilización del conocimiento. Transforma información en conocimiento reutilizable.	Responsabilidades Gestionar el ingreso de información al sistema. Todo elemento externo debe atravesar este contexto antes de incorporarse al dominio.	Responsabilidades Gestionar los activos reutilizables del sistema. Un Asset representa valor acumulado que permanece más allá de un proyecto.	Responsabilidades Gestionar el propio funcionamiento del POS. No administra trabajo. Administra el sistema.
Entidades principales - Operation - Project - Task - Deliverable	Entidades - Knowledge - Resource - Discipline	Entidades - Inbox - Idea - Resource	Entidades - Asset - Deliverable - Knowledge	Entidades - Context - Workspace - Tool
Procesos - Execute - Deliver - Review	Procesos - Learn - Build	Procesos - Capture - Process	Procesos - Build - Archive	Procesos - Configuration - Review - Governance
Produce - Deliverables - Estados operativos - Eventos de ejecución	Produce - Knowledge - Assets - Capacidades	Produce - Entidades clasificadas - Información organizada	Produce - Assets - Versiones - Capacidades reutilizables	Produce - Configuración - Convenciones - Estado del sistema

Relaciones entre Contextos



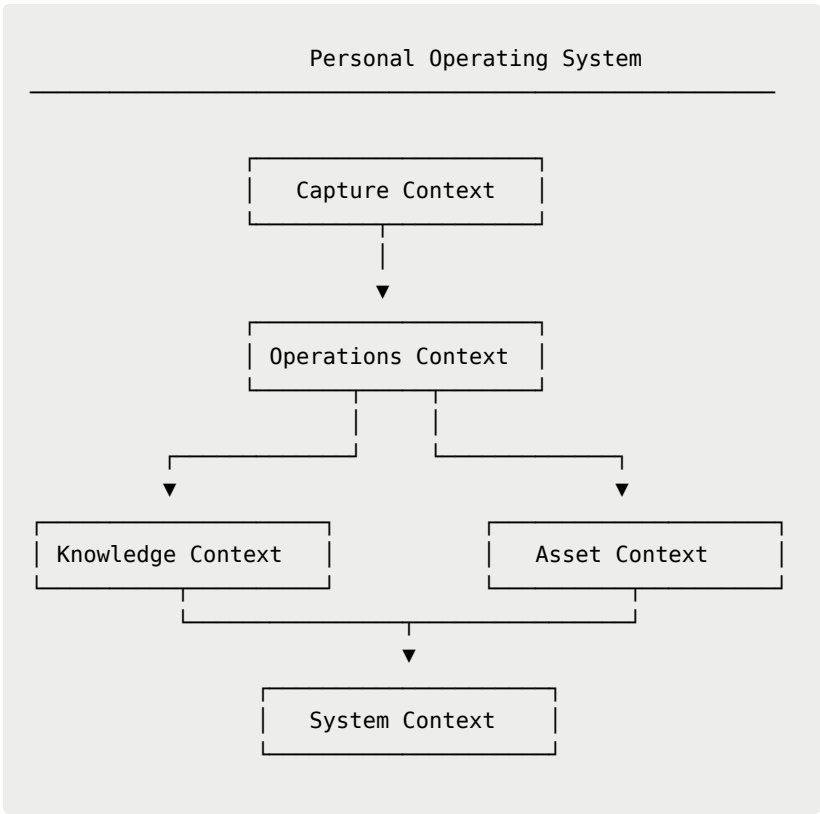
Responsabilidades

Context	Responsable de	No responsable de
Capture	Incorporar información	Ejecutar trabajo



Context	Responsable de	No responsable de
Operations	Gestionar operaciones	Almacenar conocimiento
Knowledge	Construir conocimiento	Planificar proyectos
Asset	Gestionar activos	Ejecutar operaciones
System	Mantener el sistema	Gestionar trabajo diario

Mapa conceptual



Reglas de interacción

BC-001 — Comunicación explícita

Los Contexts no modifican directamente las entidades internas de otros Contexts.

Toda interacción ocurre mediante eventos, servicios o interfaces conceptuales.

BC-002 — Lenguaje consistente

Dentro de un Context, cada entidad posee un único significado.

No existen definiciones duplicadas.

BC-003 — Bajo acoplamiento

Los Contexts conocen únicamente las capacidades públicas de otros Contexts.

Nunca sus detalles internos.

BC-004 — Alta cohesión

Las reglas relacionadas deben permanecer dentro del mismo Context.

No deben dispersarse entre múltiples Contexts.

BC-005 — Evolución independiente

Un Context puede evolucionar sin requerir cambios estructurales en los demás.

Context Mapping

Origen	Destino	Tipo de relación
Capture	Operations	Upstream
Capture	Knowledge	Upstream
Operations	Knowledge	Customer/Supplier
Operations	Asset	Customer/Supplier
Knowledge	Asset	Partnership
System	Todos	Shared Kernel (infraestructura conceptual)

Resultado esperado

Con la definición de los **Bounded Contexts**, el dominio queda dividido en áreas semánticamente independientes, donde cada una posee un modelo propio, reglas específicas y responsabilidades claramente delimitadas.

Esta separación permite que el POS evolucione sin incrementar el acoplamiento entre sus distintas capacidades, facilita la incorporación de nuevos motores y componentes, y prepara la arquitectura para definir el **Information Flow**, los **Domain Events** y un futuro diseño orientado a eventos (Event-Driven Architecture), manteniendo la coherencia del lenguaje ubicuo y del modelo conceptual ya establecidos.



Observación arquitectónica

Haría un pequeño ajuste respecto a la hipótesis inicial de los motores del *Project State*. Allí se planteaban cuatro motores: **Operations, Knowledge, Growth y System** como una hipótesis aún no formalizada. Después de haber desarrollado el Domain Model y los Architectural Elements, el dominio parece organizarse de forma más natural alrededor de **cinco Bounded Contexts** (Capture, Operations, Knowledge, Asset y System). El antiguo **Growth Engine** pierde entidad propia: el crecimiento emerge como resultado de la interacción entre **Knowledge** y **Asset**, en lugar de constituir un contexto independiente. Esto simplifica la arquitectura y reduce el solapamiento de responsabilidades.

Vistazo del Chat: Definimos **05.5 Bounded Contexts**, estableciendo los límites semánticos del dominio y proponiendo cinco contextos principales (Capture, Operations, Knowledge, Asset y System), además de ajustar la hipótesis inicial eliminando un posible solapamiento del antiguo *Growth Engine*.